TissueGraph

A blueprint for differentiable tissue: hierarchical sets, operators, and an LLM-driven simulation layer

Janelia Research Campus

June 15, 2026

Abstract

Living tissues couple biochemical signalling, cellular interaction, mechanical force, and collective dynamics across scales. The frameworks built so far (CellGraph, NeuralGraph, MPMGraph, MetabolismGraph) each implement *one* graph type; a tissue needs them coexisting. This document is a single blueprint, distilled from a working prototype, for building one repository that unifies them. It has three parts: (i) the missing primitive — a **hierarchical graph container** stated in the language of **sets** and **operators**; (ii) the **contract** that lets a large language model (LLM) drive a registry of low-level operators through a declarative *simulation* specification; and (iii) the **experiment** — ten distinct simulations and an emergent ant-trail model, all produced from the same code by changing only the simulation file — with the lessons that follow for how to construct the repository.

Part I — The abstraction

1 Two entities: sets and fields

The container is built from two kinds of object: **sets** and **fields**. A **set** S_k is a discrete collection of like entities at scale k — metabolites, particles, cells, populations, organisms (Fig. 1). Adjacent scales are linked by **parent maps**

$$\pi_k : S_k \longrightarrow S_{k+1},$$

where $\pi_k(x)$ is the higher-level object that contains x. A cell is built from two kinds of S_0 entity — its **particles** (mechanics) and its **metabolites** (chemistry) — and π_0 sends each of them to the cell it belongs to, so $\pi_0(x) = c$ reads "x belongs to cell c". The constituents of a given cell c are its *fibre*, the set of everything that maps to c,

$$\pi_0^{-1}(c) = \{\, x \in S_0 : \pi_0(x) = c \,\},$$

so a cell simply *is* the collection of its particles and metabolites. A **field** $f : \Omega \to \mathbb{R}^q$ is a continuous quantity defined on its own grid — morphogen concentration, pressure, substrate stiffness, and so on. Fields are *not* part of the containment hierarchy: they neither contain nor are contained by cells. Instead, each field is bound to exactly one set level through an *exchange* relation (particle–grid transfer, cell sampling, or population-level secretion and sensing); these couplings are the operators of Section 2.

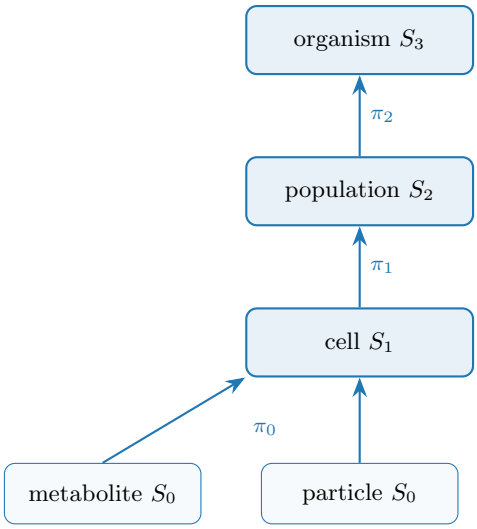


Figure 1: The containment hierarchy: the parent map π_k sends each entity to the object that contains it. A cell is the fibre $\pi_0^{-1}(c)$ — its particles and metabolites.

2 Four operators

A GNN is an **operator** $\mathcal{O}$ returning a time-derivative contribution, dispatched by the relation it acts on. We use four kinds, one per relation the container admits (Fig. 2):

Lateral	$\mathcal{O}_E,\ E \subseteq S_k \times S_k$	(ODE interaction + graph Laplacian)
Aggregate $\uparrow$	$\sum_\pi : S_k \to S_{k+1}$	(reduction over fibres of π)
Broadcast $\downarrow$	$\pi^* : S_{k+1} \to S_k$	(lift along π)
Exchange	$\mathcal{S}, \mathcal{G} : S_k \leftrightarrow F$	(scatter / gather; bipartite)

Two pieces of notation: $S_k \times S_k$ is the set of all node *pairs*, so an edge set $E \subseteq S_k \times S_k$ is simply a graph on S_k — which nodes interact (e.g. neighbours within a cutoff radius, or a fixed connectome). And $\mathcal{S}, \mathcal{G}$ are the two halves of a field coupling: *scatter* $\mathcal{S} : S_k \to F$ writes object state into the field, while *gather* $\mathcal{G} : F \to S_k$ reads it back. Each operator in turn:

Lateral $\mathcal{O}_E$. Message passing *within* one scale along the edges E. It carries the within-scale physics: pairwise interactions (forces, adhesion, signalling) plus a graph Laplacian that diffuses or smooths quantities over the edges. Examples: particle–particle elasticity, cell–cell flocking or adhesion, neuron–neuron signal propagation.

Aggregate $\sum_\pi$ (up). Pools each parent's fibre into one quantity — a cell's position is the mean of its particles, its metabolic state the sum over its molecules. This is how fine-scale dynamics are summarised into, and so move, the coarser object above.

Broadcast π^* (down). The adjoint of Aggregate: it copies a parent's state to every child in its fibre, so a decision taken at the cell level (a body force, a polarity, a signalling output) is applied identically to all of that cell's particles. Aggregate and Broadcast are a matched up/down pair on the same map π.

Exchange $\mathcal{S}, \mathcal{G}$. Couples a set to a field: scatter $\mathcal{S}$ deposits/secretes object state into the field, gather $\mathcal{G}$ samples/senses the field back onto the objects. The relation is bipartite (objects $\leftrightarrow$ grid); it is exactly the MPM particle–grid transfer (P2G/G2P) and chemotactic secretion/sensing.

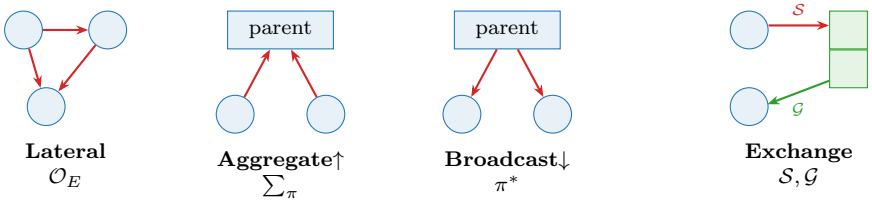


Figure 2: The four operators. *Lateral* acts within a set; *Aggregate* and *Broadcast* move across containment π; *Exchange* scatters ($\mathcal{S}$) to and gathers ($\mathcal{G}$) from a field. P2G/G2P (mechanics) and the metabolite–reaction graph are both Exchange.

3 Dynamics: a schedule

A simulation is a multi-rate composition of operators, declared as a `Schedule`. In math terms, one tick is the composition $\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$, and the dynamics is its iteration $x_{t+1} = \Phi(x_t)$ — a discrete flow (with "multi-rate" meaning some operators fire only on every k-th tick). The `Schedule` is just the code-level encoding of that composition. Each operator is *pure*: instead of editing the state in place, it only *returns its contribution* — a delta Δ_i (a time-derivative term) for the sets it touches. The schedule sums the deltas acting on each set and integrates once per tick, $x_{t+1} = x_t + \Delta t \sum_i \Delta_i$. Two consequences follow: several operators may act on the same set in one tick (their deltas simply add — none overwrites another), and because nothing is mutated in place, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The LLM searches over schedules and operator parameterizations — one well-defined action space instead of per-domain code.

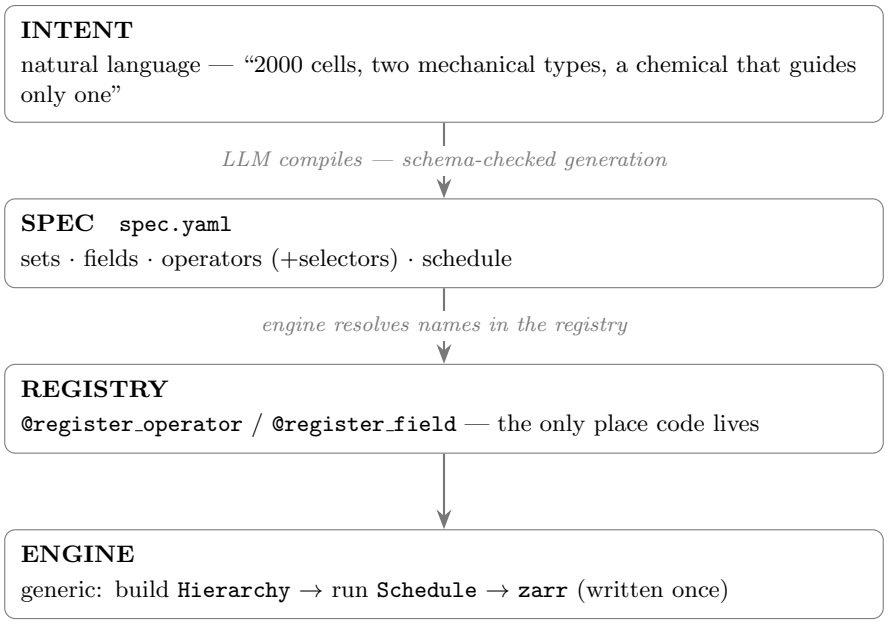
4 Glossary

Concept	Set/operator term	Symbol	Code
collection of like nodes	set (level)	S_k	`Level`
a single node	element	$x \in S_k$	row of `Level.state`
learnable per-node code	embedding	a_i	`Level.embedding`
containment	partition	$\pi_k : S_k \to S_{k+1}$	`Level.parent`
within-set links	relation	$E \subseteq S_k \times S_k$	`Level.edge_index`
continuum quantity	field	$f : \Omega \to \mathbb{R}^c$	`Field`
dynamics (GNN)	operator: ODE+Laplacian	$\mathcal{O}$	`Operator`
within-set op	lateral	$\mathcal{O}_E$	`Lateral`
up op	reduction over fibres	$\sum_\pi$	`Aggregate`
down op	lift	π^*	`Broadcast`
set↔field op	transfer	$\mathcal{S}, \mathcal{G}$	`Exchange`
the container	hierarchy	$(S_\bullet, F_\bullet, \pi_\bullet)$	`Hierarchy`
the model	composition of ops	$\mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$	`Schedule`

Part II — The LLM ↔ registry methodology

5 Two layers and the contract between them

The prototype's purpose was not the tissue physics but figuring out *how an LLM should drive a registry of low-level operators*. The answer is a strict two-layer split that meets only at one declarative, validated file — the **spec** (`spec.yaml`). In effect this is a small domain-specific language: the spec is the *source*, the registered operators are the *primitives*, and the engine is the *interpreter*.



The **spec is the contract**. The LLM owns everything above it (intent → spec); the code owns everything below (operators). They never touch directly. This makes the system both LLM-drivable and human-auditable: the spec is small, typed, and reviewable, unlike the 500-line flat `config.py` it replaces.

Being a small, typed, declarative file gives the contract the properties one wants of an interface between an unreliable generator and exacting code:

- **Verifiable.** The spec is validated against the schema *before* anything runs: an unregistered operator, a missing required property, or a malformed selector is rejected up front, so a spec that loads is guaranteed runnable (the missing-operator and missing-property cases below). Beyond well-formedness, each simulation also ships an *intent* check that measures whether the requested behaviour actually occurred — "it ran" is not "it worked".
- **Reproducible.** A run is bit-deterministic given (spec, seed) — the engine forces deterministic kernels — so the same spec yields the same trajectory, and a fresh seed draws a new realization of the same dynamics.
- **Saveable.** The spec is ∼20 lines, so it version-controls, diffs, and archives trivially; together with the seed it *is* the complete record. We archive the recipe, not the dish: store spec + seed (plus metrics and a thumbnail) and regenerate the trajectory on demand (`archive.py`, `GALLERY.md`).
- **Composable.** A variant is a base spec plus a handful of overrides, which makes ablations and the optimizer's parameter sweeps cheap.
- **Extensible.** New behaviour is one new registered operator; the operator vocabulary grows monotonically, and every earlier spec keeps working.

6 The simulation language

The contract above is, in effect, a small **domain-specific language** in declarative-data form. Like any language it has *words* (vocabulary), *grammar* (how they combine — here a typed schema, which *is* the language definition: required keys, type fractions summing to one, every operator registered, every schedule token resolving), and *semantics* (what running them means: the engine builds the `Hierarchy` and iterates the schedule, $x_{t+1} = \Phi(x_t)$). Table 1 is the complete vocabulary; every construct in it appears in the example of Listing 1.

Structurally this is an **entity–component–system** (ECS) architecture: sets are entities, their state buffers are components, operators are systems, and the schedule is the ECS system pipeline. We adopt that framing deliberately — it is the cleanest mental model and matches the engine one-to-one.

Most of the spec is plain typed key–value; the only genuine sub-grammars are the last two blocks of Table 1 — *selectors* and the *schedule*. Both are deliberately minimal but extend naturally: selectors to conjunctions and to geometric or temporal predicates (`cell[type=soft & loaded=0]`, `cell[x<0.5]`, `cell[age>10]`), and the schedule to explicit per-operator rates and nested loops.

We keep the language as *declarative data* (YAML/JSON) rather than a bespoke syntax, so an LLM can emit it under schema-constrained decoding and a human can diff, review, and archive it; the grammar is pinned by a typed schema (Pydantic or JSON-Schema), and composition, overrides, and parameter sweeps (variants, ablations, the optimizer) are delegated to a config layer such as Hydra rather than reinvented. Prior art worth borrowing: ECS engines (the scheduling model), SBML/Antimony (reaction–species vocabulary), NeuroML and MorpheusML/PhysiCell (multicellular specs), UFL (PDE operators), and NetLogo/Mesa (agent rules).

7 The simulation specification

Using the language of Section 6 (full vocabulary in Table 1), a complete simulation is a single `spec.yaml`. Listing 1 shows one: two cell `types`, `particle`s contained in each `cell`, one chemical `field`, and a four-step `schedule`. Soft cells `secrete` the chemical and `sense` (climb) it, so they self-aggregate, while stiff cells — selected out by `cell[type=soft]` — do not.

Listing 1: A complete simulation: soft cells secrete and follow a chemical and self-aggregate.

```
name: aggregate
seed: 0
n_frames: 250
sets:
  cell:
    n: 100
    types: {soft: {fraction: 0.5, youngs: 12}, stiff: {fraction: 0.5, youngs: 300}}
  particle: {parent: cell, per_parent: 100, radius: 0.02}   # containment
fields:
  chemical: {frame: grid, res: 96, diffusion: 0.1, decay: 0.05, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 15.0, rot: 0.4}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 800, drag: 4}
  - {op: secrete, at: "cell[type=soft]", to: chemical, rate: 1.0}    # only pop 1 makes it
  - {op: sense,  at: "cell[type=soft]", from: chemical, gain: 150.0} # only pop 1 follows it
schedule: [aggregate, [motility], secrete, chemical.diffuse, sense, mpm]
```

The validator (prototype: `scenario_schema.py`) guarantees that a file which loads is runnable: every `op` exists; every `at`/`to`/`from` references a declared set/field; type `fractions` sum to one;

every schedule token resolves. (One trap: never use a YAML 1.1 boolean key such as `on`/`off` — we use `at`.)

8 Operator catalog

Operators are the system's vocabulary; it grows monotonically and every past simulation keeps working. The prototype catalog:

op	kind	acts on	behaviour
`boids`	lateral	cell	collective motion (cohesion/align/separate)
`motility`	lateral	cell	active self-propulsion (wandering heading)
`adhesion`	lateral	cell	type-specific cohesion → sorting
`mpm`	exchange	particle	MLS-MPM mechanics; soft/stiff via `youngs`
`secrete`	exchange	cell→field	deposit into a field
`sense`	exchange	cell←field	chemotactic accel up-gradient
`trail`	exchange	cell	Physarum 3-sensor pheromone steering (ants)
`aggregate`	builtin	—	cell pos/vel = mean of its particles (↑)
`<field>.diffuse`	builtin	—	field self-update (Laplacian + decay)

9 Natural language → simulation: the repeatable mapping

To make the LLM's compilation *repeatable* (same request → same structure), it follows fixed rules; the most useful:

1. "N cells/agents" → a set with `n: N`.
2. "made of K X per cell" → a contained set with `parent`/`per_parent` (two levels).
3. "two types differing by P" → `types:` with P as a per-type property.
4. "mechanical/elastic/rigid" → particles + `mpm`; map stiffness to `youngs`.
5. "moves by boids / flock" → `boids`; "wanders / motile" → `motility`.
6. "creates/secretes a chemical" → a `field` + `secrete`; "follows / chemotaxis" → `sense`; "ant trails" → `trail`.
7. **"only population X does Y"** → the selector `at: cell[type=X]`. This is the highest-leverage primitive: it scopes behaviour to a subpopulation and lets two behaviours coexist.

10 When the present code is not enough (a missing model)

The pipeline fails loudly and points at the gap, e.g. for a request needing field advection or reaction–diffusion:

```
operator 'reaction_diffusion' not in registry.
Available: ['adhesion','boids','mechanics','motility','mpm','secrete','sense','trail']
```

The fix is **one new `Operator` subclass** — never an engine change: write `forward(self, H, mask) -> set: accel` (or mutate a field, return {}); decorate `@register_operator`; declare its needs; reference it in the simulation. We exercised this live twice during the experiment: adding `adhesion` unlocked cell *sorting*, and adding `trail` unlocked ant *trail-following* — each ∼20 lines, with the engine and all other simulations untouched.

11 When a cell/particle property does not exist

Operators *declare* what they need (`REQUIRES_PARAMS`, `REQUIRES_TYPE_PROPS`); the validator checks before running, resolving a property along the containment chain (`mpm` acts on particles but `youngs` lives on the parent cell's types):

```
# missing required property -> hard error, before any computation:
operator 'mpm' requires property 'youngs' on every type of 'cell'; missing on type 'soft'.
# property no operator reads -> warning (typo guard), run proceeds:
[warn] property 'viscosity' on cell.soft is read by no operator (known: ['fraction','youngs'])
```

Design rule: capabilities are declared, not discovered at a crash. The operator that consumes a property is the single place that declares the requirement, turning a would-be `AttributeError` thousands of substeps deep into a one-line message up front.

Part III — The experiment

12 A spread of simulations from one registry

Once the vocabulary existed, qualitatively different behaviours were *just different specs* over the same code — breadth comes from the intermediate representation, not from new code. Each simulation is 100–600 cells of ∼100 MLS-MPM particles (the two cell types in two colours over the shaded chemical field); eight are shown in Fig. 3:

- **aggregate** — soft cells secrete a chemical and climb it, clumping together;
- **disperse** — soft cells flee their own chemical (negative gain), spreading apart;
- **swarm** — strong boids make the cells flock and move as one;
- **wander** — active motility alone: a diffusive gas of cells;
- **sort** — type-specific adhesion segregates the two populations;
- **chase** — a fast-decaying field makes moving hotspots the cells chase;
- **crystal** — pure repulsion spaces the cells into a lattice;
- **collapse** — low diffusion with strong chemotaxis pulls soft cells into one tight cluster.

Across all eight, only the operator choices and their parameters differ; the engine, the operators, and the schedule grammar are identical.

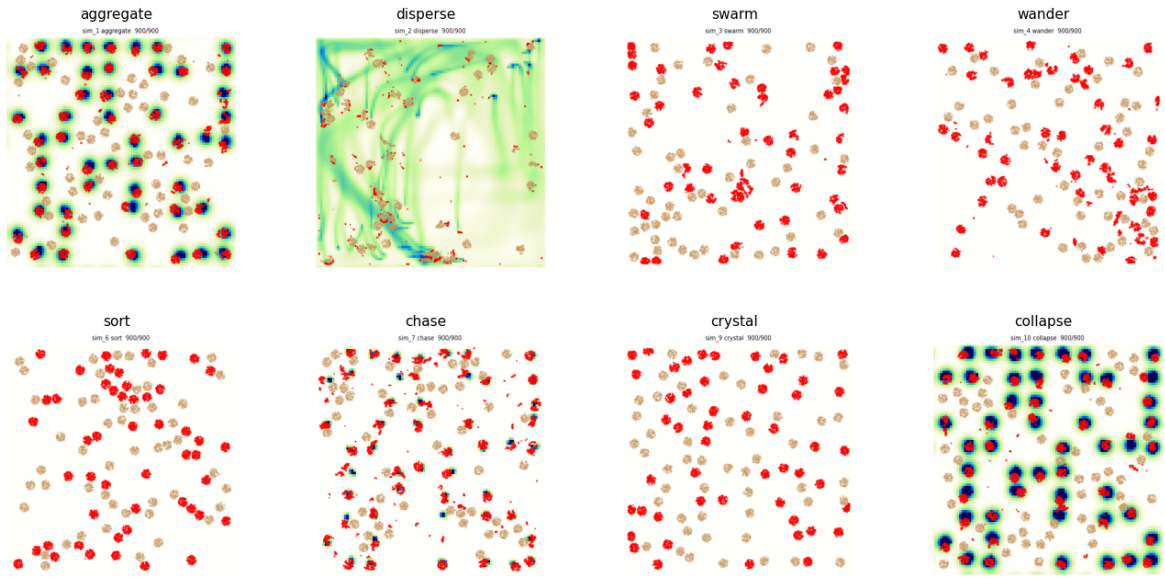


Figure 3: End states of the eight simulations described above, all generated from the same operator registry by changing only the spec.

13 Emergent ant trails

Adding the `trail` operator (a Physarum/Jones three-sensor rule: each ant samples pheromone ahead-left / ahead / ahead-right and turns toward the strongest, while `motility` drives it forward and `secrete` lays pheromone) yields self-reinforcing trails that grow, are followed, and coarsen over long runs (Fig. 4). Listing 2 is the full simulation.

Listing 2: The ant simulation: red cells lay, follow, and reinforce pheromone trails.

```
name: ant
seed: 0
n_frames: 1500
sets:
  cell:
    n: 150
    types: {soft: {fraction: 0.6, youngs: 60}, stiff: {fraction: 0.4, youngs: 300}}
  particle: {parent: cell, per_parent: 60, radius: 0.012}
fields:
  pheromone: {frame: grid, res: 160, diffusion: 0.02, decay: 0.04, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 300.0, rot: 0.06}            # persistent forward motion
  - {op: trail,  at: "cell[type=soft]", from: pheromone, turn: 0.4, sensor_dist: 0.04, sensor_angle: 0.5}
  - {op: secrete, at: "cell[type=soft]", to: pheromone, rate: 2.0}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 1200, drag: 0.6}
schedule: [aggregate, trail, motility, secrete, pheromone.diffuse, mpm]
```

14 Cells in a maze: foraging and optimization

A harder simulation exercises more of the language: two rigid cell populations start at *home* (bottom-left), navigate a walled *maze* to *food* (top-right), pick food up, and carry it back. It adds three constructs — `obstacles`, a per-cell `loaded` state, and state-gated selectors (`cell[loaded=0]`

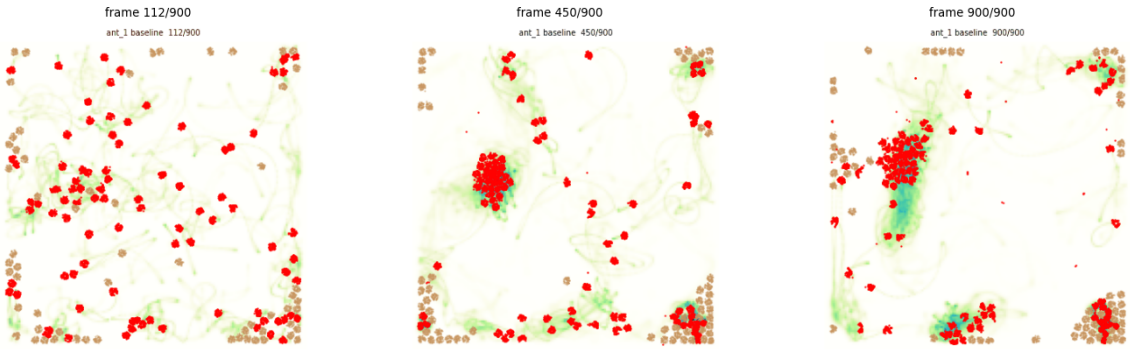


Figure 4: Ant trail-following over a long run (three frames, early to late). A diffuse network of pheromone trails (shaded) forms, reinforces, and *coarsens into a few dominant trails* with ants marching along them (right) — classic stigmergy.

seeks food, `cell[loaded=1]` returns home) — and answers the design question *how does one model an obstacle*: a wall is not a pairwise force but a **static occupancy mask reused as a boundary condition** by machinery already present. The same mask (i) zeroes the MPM grid velocity inside walls, so rigid cells cannot enter, and (ii) imposes no-flux on field diffusion, so a chemical *routes around* the walls. Two such fields — one sourced at food, one at home — become navigation gradients that solve the maze; unloaded and loaded cells climb opposite ones (Fig. 6).

Because the whole simulation is differentiable and the spec is tiny, the colony can be *optimized*: search the behavioural parameters to deliver food as fast as possible. The objective — units delivered — comes from a discrete pick-up/drop-off and is therefore non-differentiable, so we use a black-box **UCB** search over eight levers (motility speed and rotation, sense gain, MPM drag and force cap, cell stiffness and size, colony size). Over a few dozen simulations it improves delivery roughly five-fold (Fig. 7). This echoes the differentiable-simulator design literature, where a *smooth* objective is optimized by gradients through the rollout; the general recipe is **both** — UCB or CEM for the discrete and structural choices, and gradient-through-rollout on a smooth surrogate for the continuous interior.

The optimizer keeps a log of every accepted improvement (the auto-generated `WINNERS.md`, Table 2), which is itself the finding: it shows *what makes a good forager*. Two parameters move monotonically to their bounds — the colony grows to its cap ($n \approx 120$: more foragers deliver more food) and the cells soften to the floor (youngs $\to 20$: soft, deformable cells squeeze through the maze far better than the rigid ones we started from). Chemotaxis stays strong throughout (gain $\sim$ 340–400) and the cell radius sits at its minimum (smaller cells clear the gaps). The remaining knobs — motility speed and rotation, drag, force cap — are traded off rather than maximized: the best design (`winner_7`) is a balanced, moderate-speed, moderate-drag colony, not any extreme. UCB explores widely (the parameter columns are non-monotonic) while the best-so-far food rises monotonically by construction. The headline is that the search rediscovered, with nothing hand-coded, that a *large swarm of small, soft, strongly-chemotactic cells* forages a maze best.

The difference is visible in the trails themselves (Fig. 8): the initial colony barely establishes a route, whereas the optimized one lays a strong chemoattractant highway that threads both maze gaps from home to food.

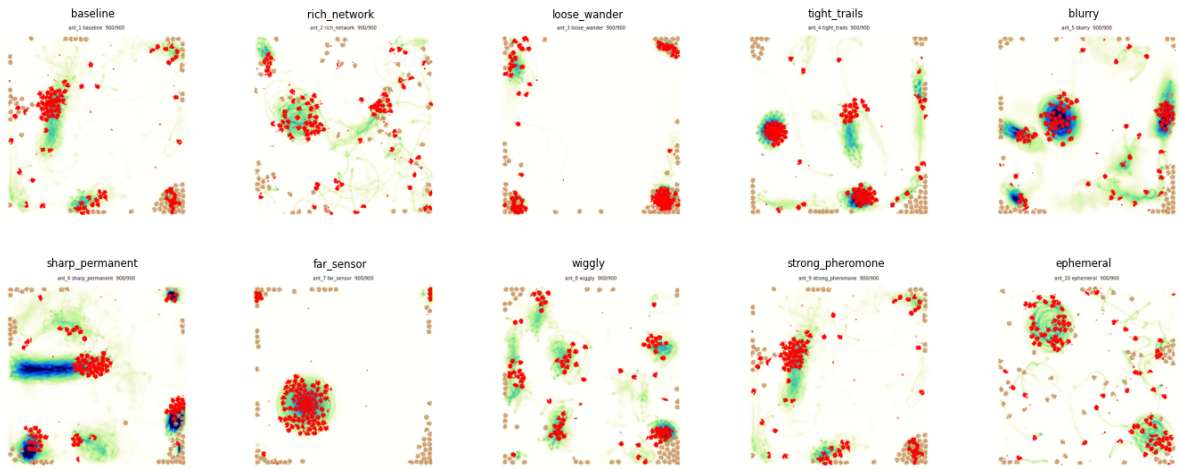


Figure 5: Ten ant variants, each the same `trail+motility+secrete` code with a few parameters changed (turn rate, sensor reach/angle, pheromone diffusion/decay, deposit rate, rotational noise). The parameter axes span the trail-formation regime — from loose wandering to tight reinforced networks.

15 What the process taught us

1. **The intermediate representation is the whole game.** Ten distinct behaviours and an ant model were all different YAML over the same code.

2. **Selectors are the highest-leverage primitive.** `cell[type=soft]` expresses "only population 1 does X" and lets behaviours coexist; every operator must honour the selector mask.

3. **Verification must check *intent*, not just "it ran".** The hard bugs were physical: unstable diffusion ($dt \cdot D > 0.25$), wrong particle volume, unclamped accelerations blowing up MPM, cell inversion at very low stiffness, GPU non-determinism breaking reproducibility, and — subtlest — the *wrong metric* (net displacement vs. nearest-neighbour clustering).

4. **Honesty about regime.** Self-secreted chemotaxis aggregates only below a density threshold (above it the field is spatially flat — correct physics); the simulation/verify output should state the regime, not silently show the one setting that worked.

5. **Determinism is a feature you must force** (deterministic CUDA kernels), or "reproduce with a new seed" is meaningless.

6. **Obstacles are boundary conditions, not forces.** A wall is one static mask reused by the MPM grid (interior zero-velocity) and the diffusing fields (no-flux); maze navigation then falls out of the field's gradient, with no special pathfinding code.

7. **Cap speed below the CFL limit.** Strong gradients otherwise drive cells to the grid's stability ceiling, where they move ballistically ("asteroids"); an explicit max-speed clamp plus overdamped drag restores smooth, biological motion.

8. **Discrete objectives need black-box search, differentiable ones admit gradients —**

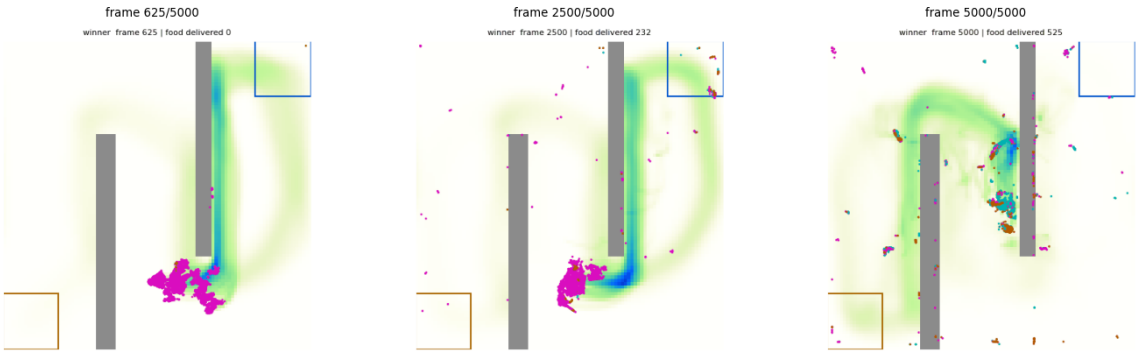


Figure 6: Maze foraging (three frames, early to late). A colony leaves home (lower-left box), threads the walls up the food gradient to the source (upper-right box), picks up food, and carries it home along the home gradient to drop it off; shading is the deposited chemoattractant.

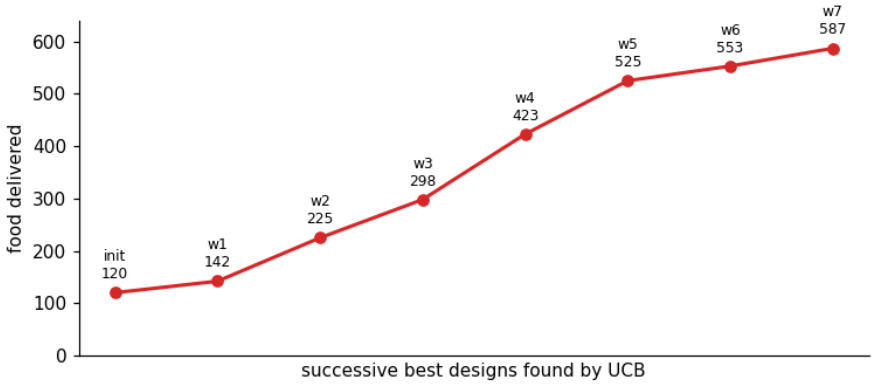


Figure 7: Optimizing the colony. Best food delivered as UCB searches the eight-parameter design space, from the initial hand-set design to the best found — a slow, soft, high-drag, many-celled colony.

use both. Food count (discrete) optimizes well under UCB; a smooth surrogate opens the gradient-through-rollout path, and the two compose: UCB for structure, gradients for the continuous interior.

9. **Reproducibility needs full-precision provenance.** Archiving rounded parameters made a winner re-simulate to a different score; the exact spec (and seed) must be stored, or the run is not bit-reproducible.

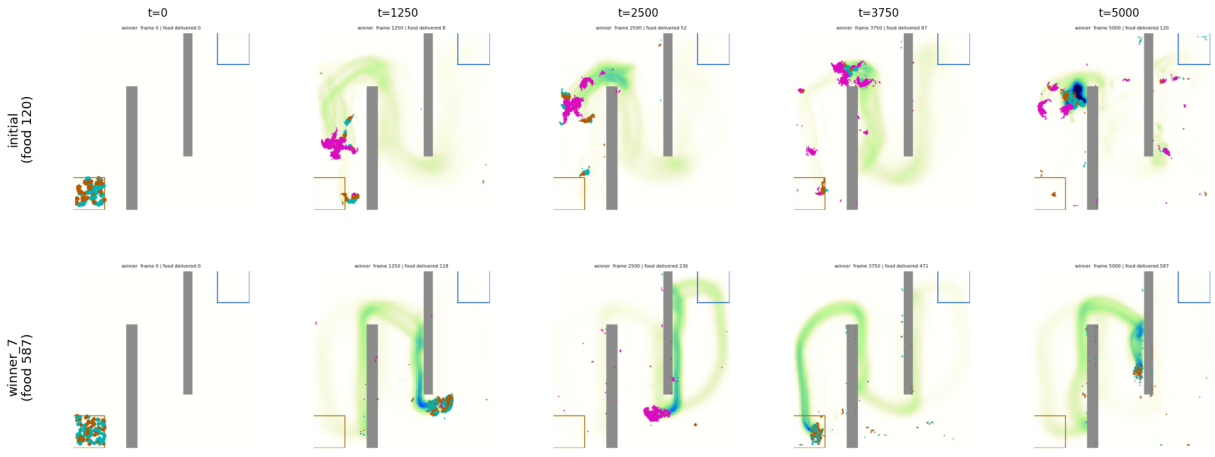


Figure 8: Initial design (top) versus the best found by UCB (bottom, `winner_7`), each a single run shown as five frames from early to late. The optimized colony forms a far more complete and persistent foraging trail.

Part IV — Building the repository

16 Proposed methodology

Layering (strict; no leaks across).

- `models/base.py` — entities, operator base, the capability-contract fields.
- `models/registry.py` — the only name→code map (entity / operator / field).
- `models/<domain>/*.py` — operators, one concern per file, each declaring `REQUIRES_*`.
- `simulation.py` — the schema + validator (the contract gatekeeper).
- `engine.py` — generic; contains **no** simulation-specific logic.
- `viz.py` — generic over the `Hierarchy`/`zarr`; swappable backends (2-D now, 3-D later).
- `archive.py` + `simulations/` — the growing, reproducible gallery.

The build loop (how LLM and code co-evolve).

1. user states intent in natural language;
2. LLM compiles → `spec.yaml` (following the mapping rules);
3. validator runs: unknown op → write one operator; missing property → fix spec or add the requirement; else it is runnable;
4. engine runs; **verify** checks intent + repeatability + well-formedness;
5. `archive.py` records spec+seed+metrics+thumb into the gallery;

6. new capability → one new registered operator (+ its `REQUIRES_*`), never an engine edit.

Invariants to hold the line.

- Code lives only behind the registry; the engine never branches on simulation names.
- Every operator honours its selector mask and declares its `REQUIRES_*`.
- Every simulation ships an intent check; "it ran" $\neq$ "it worked".
- Archive the recipe (spec+seed); regenerate the dish.
- When a result holds only in a regime, the spec/verify says so.

This is the contract that lets the LLM operate at the top (intent → simulation, plus occasional single-operator authoring) while the low-level code stays a clean, growing, verifiable registry underneath.

Construct	Semantics	Example
Top level		
`name`	run identifier	`name: aggregate`
`seed`	RNG seed — fixes determinism	`seed: 0`
`n_frames`	number of ticks to run	`n_frames: 250`
`obstacles`	static walls; a boundary condition for both the MPM grid and field diffusion	`[[.3,0,.36,.7]]`
`sets/fields/operators/schedule`	the four parts, detailed below	—
Set (S_k — an entity level / ECS entity)		
`n`	size of a top-level set	`n: 100`
`parent`	containment target of a leaf set (the map π_k)	`parent: cell`
`per_parent`	children per parent	`per_parent: 100`
`types`	named sub-populations: fraction + per-type properties	`{soft: {fraction: .5, youngs: 12}}`
Field (a continuum)		
`frame`	discretization (a grid)	`frame: grid`
`res`	grid resolution	`res: 96`
`diffusion`, `decay`	PDE coefficients	`diffusion: 0.1`
`couples_to`	the one set this field exchanges with	`couples_to: cell`
`source`	fixed source region (imposes a gradient)	`source: [.82,.82,1,1]`
Operator (a system; one per list entry)		
`op`	which registered operator to run	`op: sense`
`at`	selector — the subset it acts on	`at: cell[type=soft]`
`to`/`from`	field an Exchange writes to / reads from	`from: chemical`
⟨params⟩	operator-specific arguments	`gain: 150`
Selector (sub-grammar)		
`set`	every node of the set	`cell`
`set[attr=val]`	nodes matching `attr` (re-checked each tick)	`cell[loaded=1]`
Schedule (sub-grammar; per-tick order)		
operator name	run that operator this tick	`sense`
`[a, b]`	a group run in the same stage	`[motility, sense]`
`aggregate`	builtin: Aggregate↑ (parent = mean of children)	`aggregate`
`name.diffuse`	builtin: advance that field one step	`chemical.diffuse`

Table 1: The simulation language — vocabulary, grammar (each construct's block is where it may appear), and semantics, with examples drawn from Listing 1.

#	food	speed	gain	drag	rot	youngs	a_max	radius	n
init	120	40	180	2.00	0.30	550	700	0.01	60
1	142	74	379	2.54	0.10	517	454	0.02	46
2	225	106	253	1.25	0.31	36	599	0.01	88
3	298	83	270	0.97	0.48	78	1362	0.01	93
4	423	49	391	1.55	0.18	160	1438	0.01	106
5	525	21	341	2.99	0.40	24	1633	0.01	120
6	553	52	400	0.92	0.56	20	880	0.01	116
7	587	46	342	1.70	0.39	20	988	0.01	120

Table 2: The optimization log (`WINNERS.md`): the initial design and each significantly-better foraging design UCB found, with the eight tuned parameters. Food delivered rises from 120 to 587 ($\sim$5$\times$).